

CSCE 645 GEOMETRIC MODELING · MODULE 05 · LECTURE NOTES

Surfaces: Tensor Products and Patches

Curves in two directions, and where the construction runs out.

MODULE OUTCOMES

- Construct tensor-product Bézier and B-spline surfaces.
- Evaluate a patch and compute its normals and derivatives.
- Explain why tensor products force quadrilateral topology.
- Explain trimming and why it is necessary and awkward.
- Explain cross-patch continuity and why it is hard.

1 The tensor product construction

Given a curve scheme with basis functions B_i , the surface version applies it in two parameters over a rectangular grid of control points.

$$S(u, v) = \sum_i \sum_j B_i(u) B_j(v) P_{ij}$$

The name is apt: the basis is the tensor product of the two one-dimensional bases. Every property carries over directionally — the surface lies in the convex hull of its control grid, is affine invariant, and interpolates its corner control points when clamped.

The evaluation strategy follows from the factorisation:

$$S(u, v) = \sum_i B_i(u) \cdot \left[\sum_j B_j(v) P_{ij} \right]$$

```
vec3 eval_patch(const vec3 P[4][4], float u, float v) {
    vec3 row[4];
    for (int i = 0; i < 4; ++i)                // evaluate each row in v
        row[i] = de_casteljau({P[i][0], P[i][1],
                               P[i][2], P[i][3]}, v);
    return de_casteljau({row[0], row[1],        // then across in u
                        row[2], row[3]}, u);
}
```

A bicubic patch — 4×4 control points, degree 3 in each direction — is the standard unit, for the same reason cubics dominate curves: enough freedom for C^2 joins, little enough to stay numerically well behaved.

1.1 Derivatives and exact normals

Partial derivatives come from differentiating the basis in one direction while evaluating normally in the other. The surface normal is then the normalised cross product:

$$\mathbf{n}(\mathbf{u}, \mathbf{v}) = \text{normalize}(\partial \mathbf{S} / \partial \mathbf{u} \times \partial \mathbf{S} / \partial \mathbf{v})$$

THIS IS GENUINELY BETTER THAN A MESH NORMAL

Module 02 spent a page on how to *weight* the average of face normals, because a mesh has no normal — only an approximation whose quality depends on tessellation.

A parametric surface has an exact normal at every point, computed analytically. It does not depend on how finely you tessellate and does not change when you refine.

This matters for manufacturing (tool paths need exact surface normals), for exact offsetting, and for high-quality rendering of smooth surfaces. It is one of the real advantages that keeps NURBS in engineering use.

2 The structural limitation

A RECTANGULAR GRID OF CONTROL POINTS MEANS RECTANGULAR TOPOLOGY

The tensor product construction requires control points indexed by (i, j) over a rectangle. There is no way to have a control point with five neighbours.

So a single patch is topologically a square, and any surface assembled from patches is a quadrilateral mesh of them.

A cylinder is fine. A torus is fine. A sphere needs at least two patches and has degenerate poles where the parameterisation collapses. A character's head — which naturally wants vertices of valence 3 and 5 around the eyes, nose, and ears — does not fit the model at all.

Every remaining difficulty in this module is a consequence of this one constraint.

2.1 Trimming

Suppose you want a hole in a surface — a bolt hole in a plate. You cannot simply remove control points, because the grid must remain rectangular.

The solution is **trimming**: retain the full rectangular patch and carry a set of curves in its parameter domain marking which regions are valid. The surface is the patch restricted to the untrimmed region.

This works, and it propagates complexity into everything downstream:

- **Tessellation** must triangulate a region bounded by arbitrary curves rather than a rectangle — a constrained triangulation problem rather than a grid.
- **Adjacency** between patches is no longer structural. Two trimmed patches meant to meet along a curve have independently computed trim curves that agree only to within a tolerance.

- **Gaps.** Those tolerance mismatches become visible holes when the model is tessellated for rendering or manufacturing.

WHY CAD-TO-MESH EXPORT IS NOTORIOUSLY UNRELIABLE

Anyone who has exported a CAD model for rendering or 3D printing has encountered meshes with gaps along patch boundaries, flipped normals, and non-manifold edges.

The cause is usually trimming: the trim curves of adjacent patches do not agree exactly, so tessellating each independently produces edges that do not match. Healing these gaps is an entire category of commercial software.

Keep this in mind in Project 2 — if your input is CAD-derived, expect the repair steps of Module 02 to earn their keep.

3 Cross-patch continuity

A single patch is smooth internally by construction. Making two patches meet smoothly requires explicit conditions, and the difficulty rises sharply with what you ask for.

Continuity	Condition	Difficulty
C^0	The patches share their boundary control points.	Trivial — just use the same points.
G^1	The tangent planes agree along the entire shared edge, which constrains the row of control points adjacent to the boundary on both sides.	Manageable for two patches meeting along an edge.
G^2	Curvature agrees along the whole edge, constraining two rows on each side.	Hard. Uses up most of the available freedom, and required for reflective surfaces.
At a corner	All n patches meeting at a vertex must satisfy every edge condition simultaneously and consistently.	Often impossible at the natural degree. See below.

THE VERTEX ENCLOSURE PROBLEM

When n patches meet at a single corner, the G^1 conditions along the n shared edges are not independent: they must be mutually consistent at the shared vertex.

For certain valences — odd numbers in particular — the resulting constraint system has no solution using patches of the natural degree. The surface simply cannot be made smooth there without changing something structural.

Workarounds all cost something: raise the patch degree, split patches into more pieces, introduce small filling patches, or accept approximately- G^1 continuity and hope it is below the visible threshold.

This is the single strongest argument against patch-based modelling of organic shapes, and it is precisely what subdivision surfaces avoid — they never form explicit patches, so there is no boundary at which to impose conditions. Module 06.

4 Where patches are the right tool

Suits patches	Does not
Engineered parts: extrusions, lofts, surfaces of revolution, fillets.	Organic shapes with irregular vertices — characters, creatures, scanned anatomy.
Anything requiring exactness : manufacturing tolerances, exact conics, offsets.	Anything requiring arbitrary topology without a careful hand-built patch layout.
Analytic normals and derivatives for tool paths and simulation.	Local refinement — adding detail requires inserting an entire row or column across the patch.

Suits patches	Does not
Interchange with manufacturing and engineering software, where NURBS is the lingua franca.	Rapid iteration — maintaining cross-patch continuity by hand is slow.

The local refinement limitation is worth noting: inserting a knot in a tensor-product surface inserts an entire row or column of control points, including in regions that needed no extra detail. **T-splines** and **hierarchical B-splines** were developed specifically to allow genuinely local refinement, and they are now standard in high-end CAD.

5 Tessellation and the connection to rendering

```
// Uniform tessellation: simple, wasteful on flat regions.
for (int i = 0; i <= N; ++i)
  for (int j = 0; j <= N; ++j)
    V[i][j] = eval_patch(P, i / float(N), j / float(N));
```

Adaptive tessellation uses the same principle as Module 03: subdivide where the surface deviates from flat by more than a tolerance. For rendering, the right criterion is **screen-space** error — a patch covering ten pixels needs no more than ten pixels' worth of triangles, and CSCE 614 Module 10's warning about sub-pixel triangles applies directly.

THIS IS WHAT GPU TESSELLATION IS FOR

CSCE 641 Module 12 introduced the tessellation control, tessellator, and tessellation evaluation stages. The evaluation shader's job is exactly the computation in this module: given a patch's control points and a generated (u, v) , compute $S(u, v)$.

So the control cage becomes the vertex buffer, the tessellation factors come from a screen-space error estimate computed in the control shader, and the surface is evaluated fresh every frame at whatever resolution the current view requires.

That is the clean version of level of detail: one representation, resolution chosen at draw time, no LOD chain to author or to pop between.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
The NURBS Book — surfaces chapters	Tensor-product surfaces, trimming, and the continuity conditions, done rigorously.

Resource	What it covers for this module
Keenan Crane — DDG, surface representation material	The topological argument for why patches struggle with arbitrary surfaces.
OpenNURBS developer documentation — trimmed surfaces	How a production kernel actually represents trims and why export is difficult.
Cem Yuksel — Interactive Computer Graphics, tessellation lectures	GPU tessellation of parametric patches, connecting this module to CSCE 641.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement bicubic Bézier patch evaluation with nested de Casteljau. Render the Utah teapot from its original 32-patch control data.
- 2 Implement analytic partial derivatives and exact normals. Compare the shading against normals computed by averaging from a tessellated mesh, at coarse and fine tessellation.
- 3 Join two bicubic patches with C^0 continuity, then with G^1 . Render both with an environment map and capture the difference at the seam.
- 4 Attempt a G^1 join of three patches at a shared corner and document where the constraint system becomes over-determined.
- 5 Implement uniform and adaptive patch tessellation. Compare triangle counts at matched visual quality for a curved patch and a nearly flat one.
- 6 Implement a simple trimmed patch: a bicubic with a circular hole defined by a trim curve in parameter space. Triangulate the valid region and render it.
- 7 Tessellate two adjacent trimmed patches independently and measure the gap between their boundaries. Explain where it comes from.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the tensor-product surface formula and describe how evaluation factorises.
- 2 Why can a parametric surface give exact normals when a mesh cannot?
- 3 What topology does the tensor-product construction force, and name three consequences.
- 4 What is trimming, why is it necessary, and what does it make difficult?
- 5 State the vertex enclosure problem and say why it motivates subdivision surfaces.

- 6 Why is local refinement awkward for tensor-product surfaces, and what addresses it?
- 7 How does GPU tessellation relate to the content of this module?